



NexusNTU

Test Plan

Version 1.0

2nd November 2025

Prepared by Team CtrlAltElite

Shen Jia Cheng

Vu Thao Nguyen

Akshay Krishna

Aishwarya Anand

Nguyen Phuong Linh

Tran Son Viet

VERSION HISTORY

Version #	Implemented By	Revision Date	Approved By	Approval Date	Reason
0.1	Shen Jia Cheng Akshay Krishna	2/11/2025	Aishwarya Anand	23/10/2021	Initial Test Plan Draft
1.0	Shen Jia Cheng Vu Thao Nguyen Akshay Krishna Nguyen Phuong Linh Tran Son Viet	4/11/2025	Aishwarya Anand	30/10/2021	Final Test Plan

VERSION HISTORY	2
Test Plan Identifier.....	4
Introduction.....	4
Test Items (Functions).....	4
Software Risk Issues.....	5
Features to be Tested.....	5
Features not to be Tested.....	6
Approach (Strategy).....	6
Item Pass/Fail Criteria.....	7
Suspension Criteria and Resumption Requirements.....	9
Test Deliverables.....	9
Remaining Test Tasks.....	10
Environmental Needs.....	10
Staffing and Training Needs.....	10
Responsibilities.....	10
Schedule.....	11
Planning Risks and Contingencies.....	11
Approvals.....	12
References.....	12

Test Plan Identifier

Test Plan ID	NexusNTU-TP-001-v1.0
Level	Master Test Plan
Related Software Level	System Level (NexusNTU v1.0)
Author	Shen Jia Cheng, Akshay Krishna

Introduction

Purpose

This Master Test Plan defines the overall testing strategy, scope, resources, and schedule for the NexusNTU web application. The plan ensures that all functional and non-functional requirements specified in the System Requirements Specification (SRS) are thoroughly validated before release.

Scope

This test plan covers system-level and integration testing for NexusNTU v1.0. The testing effort focuses on validating the Progressive Web App (PWA) across mobile and desktop browsers, ensuring all core features function correctly and meet user expectations for international students at NTU.

References

- NexusNTU System Requirements Specification (SRS)
- NexusNTU Project Plan
- NexusNTU Risk Management Plan
- NexusNTU Software Quality Assurance Plan

Objectives

- Verify that all functional requirements defined in the SRS are correctly implemented
- Validate system integration with Firebase, Mapbox, and Dialogflow APIs
- Ensure all deep links to official NTU systems function correctly
- Confirm the application meets usability standards for international students
- Validate security mechanisms
- Identify and document defects to acceptable thresholds before production release
- Achieve minimum 80% code coverage for critical modules (authentication, navigation)
- Ensure WCAG AA accessibility compliance

Test Items (Functions)

Functional Testing	
ID	Item
1	User Registration and Authentication
2	Centralised Information Hub
3	Campus Navigation
4	Nearest Amenities Finder
5	AI Chatbot
6	Quick Links Dashboard
7	Currency Converter
Non-functional Testing	
ID	Item
8	Performance and Load
9	Security and Privacy
10	Usability and Accessibility
11	Acceptance testing
12	Data Integrity

Software Risk Issues

The NexusNTU project faces several identified risks that directly impact testing priorities. High-severity risks include dependency on external APIs such as Mapbox, Firebase Authentication, and Google Dialogflow, where service unavailability could disrupt core functionality. To mitigate this, testing will verify that fallback mechanisms display cached data appropriately and that deep links remain functional when external

services are degraded. Another significant risk is broken deep links to official NTU systems, which requires thorough link verification testing and validation of the user reporting mechanism.

Medium-severity risks include the potential for scope creep leading to incomplete feature implementation, necessitating strict adherence to the requirements traceability matrix during validation testing. Team member unavailability could impact test coverage, making comprehensive documentation and knowledge sharing essential. Security breaches, while lower probability due to Firebase Authentication usage, remain high-impact and require rigorous security testing including penetration testing of authentication flows and validation that no sensitive personally identifiable information is stored in the system. The project's time-boxed nature as a course deliverable means that any testing delays could compromise the final release, making efficient test execution and defect management critical.

Features to be Tested

The following is a list of features to be tested from the user's perspective of what the system does.

Feature	Level of Risk	Explanation / Testing Focus
User Registration	High	Critical for new user access.
Login	High	Required for users to access the application's main features
Campus Navigation and Route Generation	High	Core feature of the application
AI Chatbot Query Processing	High	Core feature for new students. Test accuracy, routes, and permissions.
Quick Links Dashboard	Medium	Supports daily use. Test link validity, role visibility, and content updates.
Nearest Amenities Finder	Medium	Aids convenience. Test distance accuracy, filtering, and favorites.
Centralised Information Hub	Medium	Delivers updates. Test content freshness, source sync, and ordering.

Currency Converter	Low	Utility feature. Test rate accuracy and calculation precision.
Profile Management	Low	Not crucial for main functionality, mainly used for backup of data and updating game high scores

Features not to be Tested

The following is a list of features that will not be included in the test suite in this version of the test plan.

Feature	Explanation
Storage and Processing of Sensitive Personal Data	NexusNTU is designed to exclude sensitive personal data (e.g., passport numbers, FIN, or Student's Pass details). Testing will confirm that such data is never collected or stored, but compliance testing for regulated data types is not applicable.
Direct Integration with Proprietary NTU Back-End Systems	The system uses deep-linking to official NTU systems rather than direct API integration due to lack of access to proprietary databases. Testing will not cover the internal behavior of NTU's Student Information Systems, Omnibus shuttle back-end, or Fullerton Health systems. Only deep-link functionality and transition experience will be validated.
Third-Party Service Internal Operations	External services (e.g., Firebase Authentication, Mapbox APIs, Google Dialogflow) are treated as trusted black boxes. Internal service functionality will not be tested; only integration points, error handling, and adherence to documented APIs will be validated.

Approach (Strategy)

The testing strategy for NexusNTU employs a multi-layered approach aligned with the Agile development methodology and Object-Oriented Rapid Prototyping method specified in the project plan. Testing activities are integrated throughout each sprint to enable early defect detection and continuous quality improvement. The strategy emphasizes both verification that the system is built correctly according to technical specifications and validation that it meets the intended user needs of international students.

Testing will be conducted at multiple levels starting with unit testing of individual components, progressing through integration testing of module interactions, advancing to system testing of the complete application, and culminating in user acceptance testing with representative international students. This progressive approach ensures that defects are identified and resolved at the earliest possible stage, reducing overall project risk and rework effort. The Quality Assurance team will maintain independence from development while collaborating closely to understand implementation details necessary for effective test design.

White Box Testing

White box testing techniques will be applied to the ChangePassword page component and the registrationauth controller class, as documented in the Black Box and White Box Testing documentation. This approach provides visibility into the internal logic and control flow of these critical security components. For the verifyCurrentPassword and handlePasswordChangeSubmit methods, basis path testing will be conducted using control flow graphs to ensure all decision paths are executed and validated. Each basis path identified in the control flow analysis will have corresponding test cases that verify correct behavior when conditions are met, when validation fails, and when error states are encountered.

The white box testing approach enables the team to achieve code coverage targets and identify unreachable code or logic errors that might not be apparent through black box testing alone. Test cases are designed to exercise specific paths through the code including successful password verification and change workflows, handling of empty input fields, management of password mismatch conditions, enforcement of password strength requirements, and proper handling of server errors during password update operations. This structural testing provides confidence that the authentication and security logic behaves correctly under all programmed conditions.

Black Box Testing

Black box testing techniques, specifically equivalence class partitioning and boundary value analysis, will be applied to test the registrationauth controller without requiring knowledge of its internal implementation. For the evaluatePasswordStrength method, boundary value testing will validate the scoring thresholds that categorize passwords as None, Very Weak, Weak, Good, or Strong. Test cases will exercise values at and around each boundary to ensure the classification logic handles edge cases correctly, including scores at the transitions between categories and extreme values at both ends of the valid range.

For the regauth method governing new account registration, equivalence class testing will partition input combinations into valid and invalid classes. Valid equivalence classes include scenarios where usernames do not exist in the database and passwords meet strength requirements and match confirmation entries. Invalid equivalence classes cover username conflicts with existing accounts, password mismatches between entry and confirmation fields, and passwords that fail to meet minimum strength criteria. Each

equivalence class will be tested with representative values to ensure the system responds with appropriate HTTP status codes and error messages, providing users with clear feedback on registration problems while successfully creating accounts when all conditions are satisfied.

Item Pass/Fail Criteria

Functional testing			
ID	Item	Pass criteria	Fail criteria
1	User Registration & Authentication	Account creation succeeds with valid inputs. Login and 2FA work correctly. Profile updates and password reset succeed.	Registration or login fails with valid inputs. 2FA or password reset broken. Profile changes not saved.
2	Centralised Information Hub	NTU announcements appear within 24h. Country news displays correctly. Content ordered chronologically. Deep links work.	Announcements delayed/missing. Wrong country news. Incorrect content order. Broken or wrong links.
3	Campus Navigation	Map loads NTU campus correctly. GPS accurate within 10m. Routes valid and clear. Controls respond within 2s.	Map fails or shows wrong layout. GPS inaccurate. Invalid/confusing routes. Controls lag over 2s.
4	Nearest Amenities Finder	Results show nearby amenities within 2km. Sorted by proximity. Filters accurate. Favorites persist.	No/incorrect results. Wrong sorting or filtering. Map/list views fail. Favorites lost.
5	AI-Powered Chatbot	85%+ accuracy on in-scope queries. Maintains context. Handles text/image input. Clear fallback for out-of-scope queries.	Accuracy <85%. Loses context. Fails to process input. Crashes or poor error messages.

6	Quick Links Dashboard	All hubs show correct links. School Hub personalizes properly. Role-based visibility is accurate. Deep links are correct.	Missing/wrong links. Personalization fails. Role access incorrect. Broken or wrong links.
7	Currency Converter	Converts correctly between SGD and home currency. Rates current and accurate. Completes within 2s.	Wrong conversions. Outdated rates. Slow (>2s) or failed conversions. Incorrect display.
Non-functional testing			
ID	Item	Pass criteria	Fail criteria
8	Performance & Load	First load ≤ 5 s. Subsequent pages ≤ 2 s. 98% uptime. Handles concurrent users smoothly.	Load times exceed limits. Uptime <98%. Crashes or lags under load.
9	Security & Privacy	HTTPS enforced. JWT tokens refresh/expire correctly. Secure OTP/2FA. Input validation and rate limiting effective. No PII stored.	HTTP allowed. JWT errors. Security flaws in OTP/2FA. Injection risks. PII stored. No rate limiting.
10	Usability & Accessibility	Meets WCAG AA. Keyboard navigation works. Responsive on all devices. Clear validation and accessible help.	Fails WCAG AA. Broken navigation. Layout issues. Poor validation or inaccessible help.
11	Data Integrity	Transactions complete or roll back. Validation prevents bad data. Error handling stable. Consistency under concurrency. Backup/restore works.	Partial commits. Invalid data saved. Crashes on DB errors. Data conflicts. Backup/restore fails.

Suspension Criteria and Resumption Requirements

Testing activities will be suspended if any of the following critical conditions occur, as they prevent meaningful execution or risk the integrity of test data:

- Environment Unavailability
 - Server outages, network failures, or cloud disruptions lasting more than four hours.
 - Suspension remains in effect until the infrastructure is fully restored and verified operational.
- Critical Blocking Defects
 - Defects that prevent execution of a significant portion of test cases, such as:
 - Authentication failures preventing user login.
 - Database corruption impacting multiple modules.
 - Testing will resume only after such blocking defects are resolved and verified.
- Major Requirement Changes
 - Approved mid-cycle requirement changes that invalidate existing test cases or necessitate significant test plan updates.
 - The QA Manager and Project Manager will assess the change impact and determine when testing can resume.
- Test Data Corruption or Loss
 - Test data becomes corrupted or unavailable to the extent that valid execution cannot continue.
 - Testing will pause until the test database is restored to a known good state.

Testing may resume only after all suspension conditions have been addressed and verified by the QA Manager:

- For Infrastructure Issues: Environment restored and validated through smoke testing to confirm critical functionality is operational.
- For Blocking Defects: Defects fixed and verified via regression testing to ensure no new issues were introduced.
- For Requirement Changes: Updated test cases reviewed, approved, and communicated to the test team before execution resumes.

Test Deliverables

The deliverables of this test plan include:

- Test plan document
- Test cases and requirement test cover report
- Test tools and outputs
- Error logs and execution logs
- Problem reports and corrective actions

Remaining Test Tasks

For future versions and subsequent releases of NexusNTU, the following tests will be conducted:

- User Acceptance Testing (UAT) with international student representatives to validate usability and gather feedback on user experience improvements
- Functional Testing on New Features such as event booking integration, accommodation finder, or work permit information hub
- Performance Testing under peak usage scenarios (e.g., semester start when many students access the app simultaneously)
- Regression Testing following each new feature release to ensure existing functionality remains intact
- Integration Testing with additional third-party services as the platform expands (e.g., new university systems, payment gateways)
- Mobile-specific Testing for iOS and Android native app versions (if developed beyond PWA)

Environmental Needs

The following infrastructure and services must be operational during testing to ensure valid test execution:

- Firebase Authentication, Firebase Realtime Database, and Firebase Hosting must be online and fully functional to support user registration, login, and data persistence testing
- Google Maps Platform (Mapbox as primary; fallback to Google Maps) must be accessible for Campus Navigation testing; Google Dialogflow API must be operational for AI Chatbot testing
- Stable, high-speed internet connection to simulate realistic user conditions and perform load testing
- Access to multiple modern web browsers (Chrome, Firefox, Safari, Edge) and devices (desktop, tablet, mobile) across iOS and Android for responsive design and cross-platform compatibility testing
- VPN/Proxy (if applicable): To test deep links and geographic restrictions for international students
- Monitoring and Logging: Real-time logging and monitoring systems to capture errors, performance metrics, and user behavior during testing

Staffing and Training Needs

The Lead Developer will conduct a comprehensive briefing on the PWA structure, API integrations, MVC design pattern, and Firebase backend to enable the QA team to write effective test cases and understand system behavior

All QA team members will receive training on the following:

- Training on NTU systems, international student needs, and use cases to create realistic and meaningful test scenario
- Understanding how to validate deep links to official NTU systems and verify fallback behavior when external services are unavailable
- Hands-on training on testing Firebase Authentication, Mapbox APIs, and Dialogflow integration to validate proper error handling and data flow

During the User Acceptance Testing (UAT) phase, international student participants will receive minimal guidance to allow the QA team to observe natural user behavior, identify pain points, and gather authentic feedback on the application's usability and effectiveness. This approach will help identify areas for improvement that might not be apparent from standard functional testing.

The QA Manager and Lead Developer will be responsible for providing initial training. Subsequently, the QA Lead Engineer will mentor other team members and ensure knowledge transfer throughout the testing phases.

Responsibilities

The following is the list of responsibilities related to the Test Plan, their description, and the assigned team member in charge of assuring the responsibility is carried out.

Responsibility	Description	Assignee
Risk Identification and Assessment	Identifying, documenting, and assigning risk levels to all functional and non-functional test items; maintaining the risk register.	Shen Jia Cheng
Feature Selection and Prioritization	Identifying core features of NexusNTU to be tested; prioritizing based on risk level and business criticality; ensuring all SRS requirements are mapped to test cases.	Vu Thao Nguyen
Test Strategy and Approach	Designing the overall testing methodology, test	Akshay Krishna

	levels (unit, integration, system, UAT), and testing techniques (white box, black box); ensuring alignment with Agile methodology.	
Test Tool and Environment Management	Ensuring all required testing tools (Selenium, Jest, Postman, Axe), software, and hardware are available and functional; managing test environment configurations and database instances.	Aishwarya Anand
Schedule Management and Conflict Resolution	Coordinating with the Project Manager and Lead Developer to maintain the testing schedule; resolving scheduling conflicts promptly; managing buffer time allocation in case of delays.	Nguyen Phuong Linh
Training and Knowledge Transfer	Conducting training sessions on testing tools, NexusNTU architecture, API testing, and UAT procedures; ensuring all team members are equipped to execute the test plan effectively.	Tran Son Viet
Critical Test Item Decisions	Evaluating test items not explicitly covered in the test plan; making final decisions on whether additional testing is required; documenting rationale for decisions.	Aishwarya Anand
Test Execution and Defect Logging	Executing test cases, documenting test results, logging defects in a centralized tracking system, and	All QA Team Members

	communicating issues to the development team.	
Traceability and Reporting	Generating test execution reports; tracking code coverage metrics; reporting overall test status to stakeholders.	Vu Thao Nguyen

Schedule

The testing schedule is aligned with the overall project plan according to the Gantt chart maintained by the Project Manager. The schedule is structured to allocate sufficient time for comprehensive testing across all phases:

1. **Phase Allocation:** Unit and integration testing begin immediately upon component completion; system testing commences once all components are integrated; UAT is scheduled in the final 2 weeks before production release
2. **Buffer Week:** One week of buffer time is allocated following the planned testing end date to accommodate unforeseen delays, retesting of fixed defects, or scope adjustments
3. **Weekly Synchronization:** The QA Manager conducts weekly check-ins with the Lead Developer to track component development progress and ensure components align with the test plan timeline
4. **Agile Sprints:** Testing activities are integrated into each sprint cycle, allowing continuous defect detection and early feedback to the development team

Planning Risks and Contingencies

The following are the identified risks that may impact the Test Plan and suggested contingencies should they occur.

Risk	Contingency
External API Unavailability (Firebase, Mapbox, Dialogflow)	Pre-arrange with service providers for test environment stability; implement timeout and fallback testing scenarios; use API mocking tools (Mockito, Jest mocks) to test error handling independently
Late Delivery of Components	Allocate buffer week in schedule; parallelize testing of independent components; conduct unit testing on partially completed modules; maintain

	detailed readiness criteria for each component
Requirement Changes Mid-Cycle	Establish change control process with Project Manager; update RTM and test cases before resuming testing; use buffer week to accommodate changes without delaying release
Performance Issues Under Load	Conduct load testing using tools like JMeter early in the testing phase; identify bottlenecks and communicate to development team; plan for optimization sprints if needed

Approvals

Following completion of final testing, the QA team will deliver a comprehensive Test Summary Report that includes:

- Overall test execution status (% tests passed/failed)
- Severity breakdown of identified defects and their resolution status
- Code coverage metrics for critical modules
- Compliance confirmation (WCAG AA accessibility, security standards)
- Risk assessment of known unresolved issues (if any)
- Recommendation for production release

Stakeholder	Signature
Quality Assurance Manager	
Project Manager	
Client	

References

- NexusNTU System Requirements Specification (SRS) v1.0
- NexusNTU Project Plan and Gantt Chart
- NexusNTU Risk Management Plan
- NexusNTU Software Quality Assurance Plan
- NexusNTU Test Cases and Requirement Test Coverage Plan v1.0